

一、破冰与愿景：AI 编程从“文本接龙”到“闭环控制”

核心问题：从 Chat 到 Agent，AI 编程发生了什么质变？

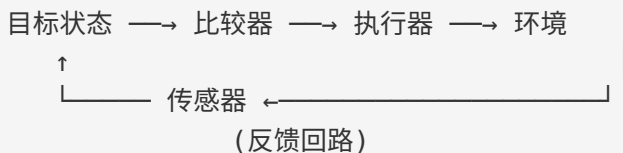
1.1 反馈的力量：从维纳的控制论到 Boyd 的 OODA

控制论：一切闭环系统的思想源头

“闭环控制”这个概念的知识源头，要追溯到 1948 年。

数学家 Norbert Wiener 出版了《控制论：或关于在动物和机器中控制和通信的科学》（Cybernetics），两年后又写了面向大众的《人有人的用处》（The Human Use of Human Beings）。Wiener 在二战期间研究防空火控系统时发现了一个跨越学科的核心原理：**任何有效的控制系统——无论是恒温器、防空炮、还是人体的运动神经——都依赖同一个机制：将输出的结果反馈回输入端，与目标进行比较，据此调整下一步行动。**

这就是**负反馈回路（negative feedback loop）**。Wiener 用一个简洁的公式概括：



没有反馈的系统是“开环”的——射出去的炮弹不会自己修正弹道。有反馈的系统是“闭环”的——导弹能持续追踪目标调整航向。**开环系统只能靠初始精度，闭环系统能持续纠偏。**这个区别在简单场景中不明显，但在复杂、动态、充满不确定性的环境中，是能力的分水岭。

Wiener 的控制论影响了此后几乎所有涉及“自适应”和“自动控制”的领域——从工业自动化到认知科学，从经济学的动态博弈到生物学的稳态调节。

Boyd：将控制论推进到对抗环境

John Boyd 是美国空军历史上最具影响力的军事理论家之一，但他既不是将军，也不是学者——他是一名战斗机飞行员和战术教官。

Boyd 在 1950 年代作为 F-86 飞行员参加过朝鲜战争，后来在内利斯空军基地担任战术教官，号称“40 秒 Boyd”——他敢跟任何飞行员打赌，从任何不利态势起手，40 秒内反转到攻击位置。据说从未输过。

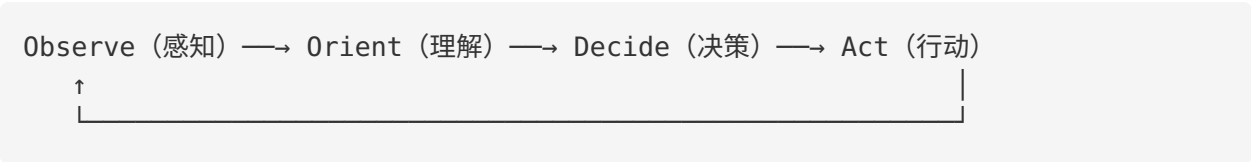
1960 年代，Boyd 从大量空战数据中发现了一个反直觉的现象：**决定空战胜负的往往不是飞机的纸面性能参数（速度、升限、火力），而是飞行员感知环境和快速响应的能力。**他分析了不同机型的设计差异——

设计维度	有利于快速决策的设计	不利于快速决策的设计
座舱视野	气泡式全景座舱（360° 环视）	窄窗式座舱（视野受限）
操纵系统	液压助力（轻推即响应）	钢索机械联动（费力、有延迟）
对飞行员的意义	感知更快、行动更快	决策再好也来不及执行

一架飞机可以爬得更高、飞得更快、火力更猛——但如果飞行员看不清态势、操纵响应慢半拍，纸面参数再好也发挥不出来。

越南战争期间，Boyd 驻扎泰国，进一步观察到：在真实对抗中，**快速获取信息并据此行动的能力**正在取代传统的消耗-火力模型，成为决定胜负的关键因素。

1970 年代初，Boyd 将二十年的空战分析和战场观察提炼为一个通用决策框架——**OODA 循环**：



气泡座舱加速了 Observe，液压飞控加速了 Act。更快的感知 + 更快的行动 = 整个循环转速更快。

Boyd 的核心洞察：在动态对抗环境中，循环转速比单次决策质量更重要。快速的“足够好”决策胜过迟到的“完美”决策——因为完美决策做出时，它所针对的局面可能已经不存在了。循环更快的一方持续用最新信息纠偏，更慢的一方则始终在对过时的现实做反应。几轮下来，慢的一方彻底失去态势感知——不是被打败的，是被“转晕”的。

这个理论后来的影响远超空战领域。美国海军陆战队将 OODA 循环写入作战条令，硅谷的精益创业方法论（Build-Measure-Learn）是 OODA 的商业变体，敏捷开发的短迭代周期本质上也是在加速 OODA 循环。**在任何需要应对不确定性的领域，循环转速都是核心竞争力。**

从 Wiener 到 Boyd，一条清晰的知识谱系浮现出来：

Wiener 控制论 (1948)	Boyd OODA (1970s)	Yao ReAct (2022)
负反馈回路	→ 对抗环境下的决策循环	→ LLM 的推理-行动循环
传感器→比较器→执行器	Observe→Orient→Act	
Observation→Thought→Action		
核心：反馈纠偏	核心：循环转速	核心：闭环放大模型能力

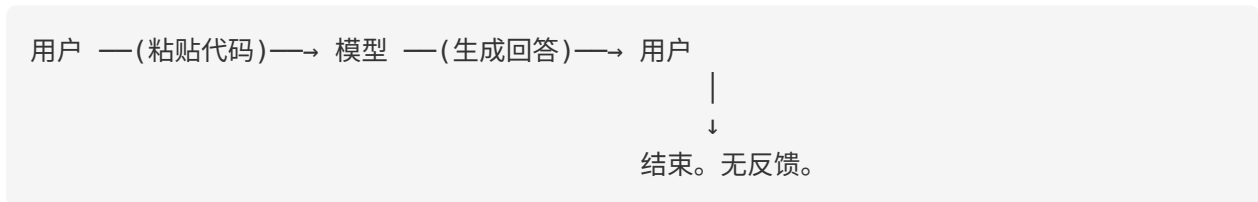
Wiener 发现了反馈的力量。Boyd 发现了循环转速的力量。2022 年的 ReAct 论文，则将这两个洞察带入了大语言模型的世界。

1.2 从空战到编程：开环生成 vs 闭环控制

Boyd 的理论映射到 AI 编程领域，惊人地精确。

Chat 模式 = 开环系统

传统的 Chat 模式本质上是一个**开环系统 (open-loop system)**：用户把代码粘贴给模型，模型做一次推理，生成一段修复建议，然后——就没有然后了。模型不知道自己的建议是否正确，不知道代码能不能跑，甚至不知道这个文件在磁盘上的什么位置。

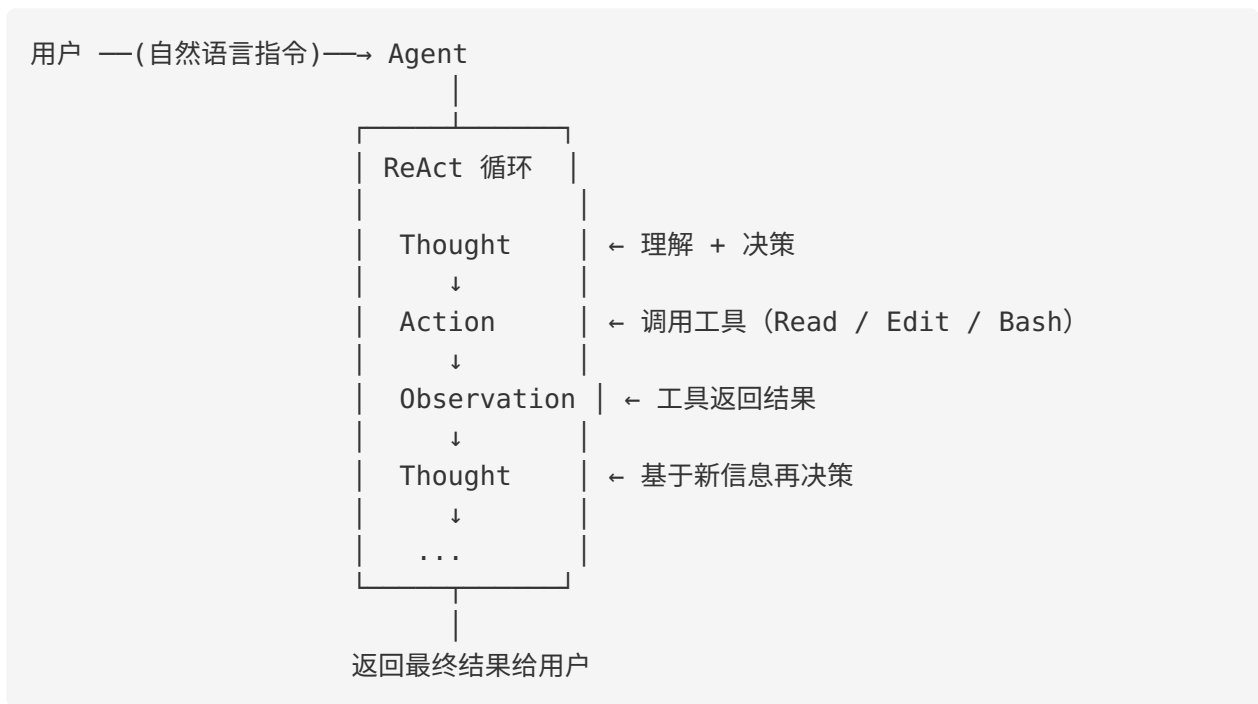


这就像 Boyd 分析的那种窄窗座舱 + 钢索飞控的飞行员：做一次判断，等着看结果。如果判断错了——抱歉，已经来不及了。

开环系统有一个根本性缺陷：**错误无法被自动发现和纠正**。模型可能生成了一段看起来合理但实际引入新 bug 的代码——它自己不知道，用户如果不仔细验证也不知道，直到上线出事。

Agent 模式 = 闭环系统

Agent 模式完全不同。模型不再是一个只会“说”的嘴巴，而是一个有**感知能力**（读文件、搜索代码）、**行动能力**（修改文件、执行命令）和**验证能力**（运行代码、检查输出）的完整系统。



每一步 Action 都产生 Observation（工具执行的真实结果），每个 Observation 都驱动新的 Thought。这个循环持续旋转，直到模型认为任务完成。

这就是闭环。从 Chat 到 Agent 的质变不是“模型变聪明了”，是**系统从开环变成了闭环**。同一个模型，放在开环架构里只能猜，放在闭环架构里能试、能看、能改、能验证。

1.3 ReAct：从“只会想”到“能想也能做”

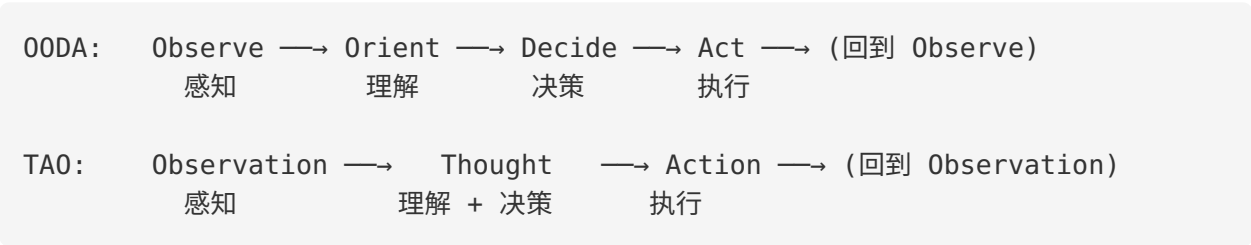
ReAct 的诞生有一个直接的前因：**Chain-of-Thought (CoT) 的局限性**。

2022 年 Wei et al. 的 CoT 论文证明了一件重要的事：让模型“一步步想”（step-by-step reasoning）能显著提升复杂推理任务的准确率。但 CoT 有一个根本性缺陷——它**只能“想”，不能“做”**。模型可以写出漂亮的推理链，但这条推理链完全基于模型的内部知识，没有与外部世界的任何交互。当推理需要事实性信息时（“这个 API 的当前版本是什么？”“这个文件第 42 行写了什么？”），CoT 只能靠记忆或猜测——而模型的记忆是有截止日期的，猜测就是幻觉。

ReAct 论文的作者 Shunyu Yao 等人（普林斯顿大学，2022）精确地指出了这个问题：**推理 (Reasoning) 和行动 (Acting) 在以往的研究中是分离的两条线——CoT 只做推理，工具调用只做行动——但人类解决问题时，推理和行动是交替进行的**。你不会先在脑子里把整个解法想完再动手，而是想一步、试一步、看到结果、再调整思路。

ReAct 的核心思想就是将这两条线合并：让模型**交替执行推理和行动**，而不是先想完再做、或者不想就做。每一步推理利用上一步行动的观察结果，每一步行动基于当前推理的结论。

ReAct 定义的 **TAO 循环**（Thought-Action-Observation）与 Boyd 的 OODA 存在精确的对应关系：



TAO 将 OODA 的 Orient（理解）和 Decide（决策）压缩为一个 Thought 步骤。这不是简化，而是对 LLM 特性的精确适配——大语言模型在一次推理中天然地同时完成理解和决策，这两个认知过程在 Transformer 的自注意力机制中是不可分割的。

ReAct 论文的真正贡献不在于这个循环有多精巧，而在于它证明了一个反直觉的结论：同一个模型（PaLM-540B），在 ALFWorld 交互决策任务上，纯行动模式（Act）的成功率只有 45%，加上 ReAct 推理循环后跃升到 71%——仅靠 1-2 个 few-shot 示例就超越了用 10⁵ 条专家轨迹训练的模仿学习基线 BUTLER（37%）（论文 Table 3）。在知识问答任务（HotPotQA/Fever）上，ReAct 与 CoT 各有胜负，但两者结合（ReAct → CoT-SC）达到最佳效果（Table 1）。模型本身没变，循环机制带来了质的提升——这就是 Boyd 的“循环机制是放大器”的活体验证。

这一发现对工程实践的指导意义是：**投入一倍的精力优化提示词（提升“模型能力”），不如投入一倍的精力优化 Agent 循环机制（提升“系统节奏”）**。这正是 Claude Code 的核心设计理念——它的竞争力不在于 Claude 模型比其他模型强多少（虽然确实强），而在于整个 Agent 系统的循环机制设计得多好。

CC 源码实证（`src/constants/prompts.ts`，~915 行）：CC 的 system prompt 不是一整段文本，而是**分段组装**的工业结构。它分为两大块，中间由 `SYSTEM_PROMPT_DYNAMIC_BOUNDARY` 标记分界：

段位	内容	是否可缓存
静态段（分界线之前）		
1. Identity + Safety	身份声明、安全指令	✓
2. System Section	输出渲染、权限模式、hooks	✓

段位	内容	是否可缓存
3. Doing Tasks	编码行为规则（先读后改、不镀金）	✓
4. Actions	可逆性/爆炸半径，危险操作清单	✓
5. Using Tools	工具使用指令（用 Read 不用 cat）	✓
6. Tone & Style	不用 emoji、简洁	✓
7. Output Efficiency	“Go straight to the point”	✓
动态段（分界线之后）		
8. Agent/Skill 指导	Agent tool 参数、skill 列表	✗
9. Auto-memory	记忆系统提示	✗
10. Environment	CWD、git status、platform、model	✗

分界线的工程意义：Anthropic API 的 prompt cache 只对 `SYSTEM_PROMPT_DYNAMIC_BOUNDARY` 之前的静态段生效。静态段在所有会话中不变（~5K tokens），一次缓存后续复用——**每轮 API 调用可节省 ~5K tokens 的重复输入成本**。动态段（CWD、git status 等）每次会话不同，无法缓存。

这个设计揭示了 harness 工程的一个常被忽视的维度：**不只是 prompt 说什么重要，prompt 的组织结构也直接影响成本**。把“不变的行为规则”放前面（利用缓存），“变化的环境信息”放后面——这是一个纯工程层面的优化，与模型能力无关，但对多轮循环中的累积成本影响很大。

LLM 与 Harness 的分工

理解了 ReAct 循环和 system prompt 架构之后，一个自然的问题是：**在整个 Agent 系统中，LLM 到底负责什么，工程代码（harness）又负责什么？**

很多人直觉上认为“Agent 的能力 ≈ 模型的能力”。但从 CC 的源码可以看到，模型只负责系统中“需要智能”的部分，大量关键工作由 harness 代码完成——模型甚至不知道这些工作的存在。

OODA 环节	LLM 负责（需要智能）	Harness 负责（纯工程）
Observe	理解工具返回的内容，识别关键信息	执行工具、截断过长输出、控制信息粒度（Read 的 offset/limit）

OODA 环节	LLM 负责（需要智能）	Harness 负责（纯工程）
Orient	从观察中推理，形成假设（“87.5=350/4，除数错了”）	触发上下文压缩、保留关键失败经验（compact 第 4 段）
Decide	选择下一步行动，决定调用哪个工具、传什么参数	—
Act	生成结构化的工具调用（tool_use JSON）	解析 JSON、权限检查、真实执行、格式化返回结果
循环控制	判断任务是否完成（end_turn）	12 个退出点、7 个继续点、max_turns 硬限制
记忆管理	生成压缩摘要（auto-compact 的 9 段结构）	决定何时触发压缩、注入恢复文件、熔断失败重试
安全边界	—（模型不知道权限系统的存在）	7 种权限模式、8 个规则来源、Hook 系统
Prompt 工程	—（模型不知道自己的 prompt 是怎么来的）	分段组装、缓存优化、动态注入环境信息

Decide 是 LLM 唯一独占的环节——harness 不参与决策。其他所有环节都是 LLM 与 harness 的协作，甚至有两个环节（安全边界、Prompt 工程）LLM 完全不参与。

这张表揭示了一个容易被忽视的事实：**Agent 系统中超过一半的关键工作是纯工程代码完成的，模型甚至不知道它们的存在**。CC 的 `queryLoop()` 有 1488 行（第五讲会详细拆解），其中模型能“感知”到的只是每轮的 `tool_use` → `tool_result` 交互——循环控制、上下文压缩、权限检查、错误恢复全部在幕后发生。

这也是为什么 Boyd 推论一说“基础能力是门槛，循环机制是放大器”——LLM 提供智能，harness 提供结构。**同一个 LLM，放在不同的 harness 里，表现可以相差 6 倍**（Meta-Harness 论文的结论，加餐环节会展开）。整门课程拆解的每一个部件——system prompt、工具设计、上下文管理、循环控制、协同编排、安全机制——都是 harness 的组成部分。

1.4 Demo 1 演示：同一个 Bug，两种修复方式

为了让这些概念从纸面走到现实，我们用一个具体的 bug 做对比演示。

演示场景

文件 `buggy_code.py` 包含一个经典的 off-by-one 错误：

```
def calculate_sum(numbers):  
    """Calculate the sum of a list of numbers"""  
    total = 0  
    for i in range(len(numbers)):  
        total += numbers[i + 1] # Bug: i+1 越界  
    return total
```

当 `i = len(numbers) - 1` 时，`i + 1` 超出列表索引范围，抛出 `IndexError`。正确写法是 `numbers[i]`。

Chat 模式的表现

用户必须自己打开文件、复制代码、粘贴到对话框。模型看到代码后，一次性给出回答：

“第 9 行 `numbers[i + 1]` 有越界错误，应该改成 `numbers[i]`。请手动修改文件并重新运行测试。”

回答看起来正确。但这里有三个关键缺陷：

1. **模型没有运行过代码** — 它不知道报错的具体信息（`IndexError: list index out of range`）
2. **模型没有验证修复** — 它不知道改成 `numbers[i]` 后输出是否是预期的 `Sum: 15`
3. **如果模型给错了建议** — 没有任何机制能发现这个错误

交互到此结束。验证工作全部推给用户。

Agent 模式的表现

用户只需要说一句话：“**tests/buggy_code.py 运行会报错，帮我修复**”。不需要粘贴代码。

Agent 进入 ReAct 循环，自主执行以下步骤：

Round	Thought	Action	Observation
1	“我需要先看看这个文件”	Read 读取 <code>buggy_code.py</code>	获得完整源码（含行号）

Round	Thought	Action	Observation
2	“第 9 行 <code>i+1</code> 有越界风险”	Edit 将 <code>numbers[i + 1]</code> 替换为 <code>numbers[i]</code>	“Successfully edited”
3	“修完了，需要验证”	Bash 执行 <code>python buggy_code.py</code>	“Sum: 15”（正确！）
4	“输出正确，任务完成”	—	<code>stop_reason:</code> "end_turn"，退出循环

Agent 完成了完整的 **Localization**（Round 1：定位 bug）→ **Repair**（Round 2：修复）→ **Validation**（Round 3：验证）闭环。每一步的 Observation 都是真实执行结果，不是模型的猜测。

对比总结

维度	Chat 模式（开环）	Agent 模式（闭环）
上下文获取	用户手动粘贴	Agent 自主读取文件
修改方式	告诉用户怎么改	Agent 直接修改文件
验证	“请手动运行测试”	Agent 自己运行并检查输出
错误发现	无（等用户发现）	Observation 暴露错误，下轮修正
工具调用	0 次	3 次（Read + Edit + Bash）
本质	单次开环文本生成	多轮 ReAct 循环（T-A-O）

一句话概括：**Chat** 是“看图猜病”，**Agent** 是“看图 → 化验 → 确诊 → 治疗 → 复查”。

如果 Live Demo 出错

用小模型做 Live 演示，模型完全可能犯错。这些错误恰好是最好的教学素材——可以当场定位到 OODA 的哪个环节失灵：

- **修错地方**（Agent 改了不相关的代码）→ Observe 环节失灵：没有正确理解文件内容

- **格式错误**（工具调用的 JSON 格式崩坏）→ Act 环节失灵：小模型生成结构化输出的能力边界
- **反复修不对**（改了还是报错，但换同样的方法再改）→ Orient 环节失灵：无法从失败中学习。后面讲 Reflexion 时会回到这个问题

失败本身就是教材。选择小模型而非前沿模型做演示，核心原因就是它的失败模式可控、可解释。一个前沿模型（如 Claude Opus）一次就修对了，观众只能鼓掌；一个小模型挣扎几轮后修对了，观众才能**看到循环机制的放大效应**——Boyd 的理论从抽象概念变成了屏幕上正在发生的真实过程。

1.5 课程双主线：任务结构与 OODA 演进

理解了开环到闭环的质变后，可以建立整门课程的认知框架。今天的三小时内容沿两条正交的主线展开：

主线一：任务结构线 — Localization → Repair → Validation

所有编程 Agent，不论架构多复杂，本质上都在做三件事：

Localization（找到改哪里）→ Repair（生成修改）→ Validation（验证对不对）

这个三阶段结构最早由 Agentless 论文（Xia et al., UIUC, 2024）显式提出。但它抽象的是一个**通用任务结构**——Claude Code 在修 bug 时走的也是 $L \rightarrow R \rightarrow V$ ，只是实现方式不同：

阶段	Agentless 实现	Claude Code 实现
Localization	固定漏斗：文件 → 函数 → 行号	动态探索：Glob → Grep → Read → LSP
Repair	多候选采样（高 temperature 生成 N 个 patch）	单次生成 + Edit 工具的 search/replace
Validation	测试套件筛选最优候选	Bash 跑测试，失败则回溯到 L 或 R

关键区别：Agentless 的 $L \rightarrow R \rightarrow V$ 是**单向流水线**（不可回溯，靠广度采样兜底），Claude Code 的 $L \rightarrow R \rightarrow V$ 是**可回溯的循环**（靠深度迭代兜底）。骨架相同，容错策略不同。这个对比在后面的“架构反思”一节会展开讨论。

主线二：OODA 演进线 — 每篇论文优化了循环的哪个环节

今天引用的论文不是随机堆砌的文献综述。它们构成一条清晰的**认知递进线**——每篇论文都在回答 OODA 循环的某个具体问题：

演进阶段	论文	优化的 OODA 环节	核心贡献
建立循环	ReAct (Yao, Princeton, 2022)	建立循环本身	从开环生成到闭环控制的范式革命
增强循环	Toolformer (Schick, Meta, 2023)	Act（行动能力）	让模型学会调用外部工具，从“只能说”到“能操作”
	Reflexion (Shinn, Northeastern/MIT, 2023)	Orient（从失败中学习）	将失败信号转化为结构化经验，注入下一轮 Thought
定义战场	SWE-bench (Jimenez, Princeton, 2023)	（评测基准）	用 2,294 条真实 GitHub Issue 衡量 Agent 端到端能力
编程领域	SWE-agent (Yang, Princeton, 2024)	Observe（感知质量）	ACI 设计控制信息粒度，解决“眼睛”的问题
	AutoCodeRover (Zhang, NUS, 2024)	Observe + Act	通过 AST 结构化感知和操作代码
	CodeAct (Wang, UIUC, 2024)	Act（行动格式）	用可执行代码替代 JSON 工具调用
架构反思	Agentless (Xia, UIUC, 2024)	质疑循环本身	固定流水线 + 多候选采样也能达到 Agent 70-80% 效果

演进阶段	论文	优化的 OODA 环节	核心贡献
多 Agent	MetaGPT (Hong et al., 2023)	编队协调（固定角色）	预定义 PM/ Architect/SWE 角色 + SOP 流程的多 Agent 协同
	AutoGen (Wu, Microsoft Research, 2023)	编队协调（对话驱动）	Agent 间通过多轮对话自组织协同，通用框架设计
	OpenHands (Wang et al., 2025)	编队协调（工业实践）	最接近工业级的开源实现，沙箱隔离 + Controller 硬编码派发

这条演进线的内在逻辑是：先建立循环（ReAct）→ 再增强循环各环节（Toolformer 增强手、Reflexion 增强脑、SWE-agent 增强眼）→ 将循环搬进编程领域并优化（SWE-bench 定义目标函数、SWE-agent/AutoCodeRover/CodeAct 分别优化 Observe 和 Act）→ 反思循环是否必要（Agentless 用最简实现挑战循环的复杂度）→ 从单体扩展到编队（MetaGPT/AutoGen/OpenHands 探索多 Agent 协同的不同范式）。每一步都不是凭空跳跃，而是对上一步暴露的瓶颈的回应。

值得注意的是，多 Agent 这三篇论文代表了三种截然不同的协同范式：**MetaGPT 是固定角色+流程驱动**（像传统企业的 SOP），**AutoGen 是对话驱动+自组织**（像开放式圆桌会议），**OpenHands 是控制器硬编码派发**（像流水线上的调度系统）。Claude Code 的 Agent Team 走了第四条路——**协调原语工具化，LLM 自主编排**，不预设固定角色，不硬编码流程，把 TeamCreate、TaskCreate、Agent、SendMessage 都做成工具，让模型在运行时自己决定怎么拆、怎么分、怎么合。这个设计选择的深层原因和权衡分析，在 Agent Team 模块会展开。

二维认知地图

两条主线交叉构成一张二维空间。11 篇论文散布其中——这张图就是今天三小时的 GPS：

	Localization	Repair	Validation
Observe	SWE-agent AutoCodeRover		

Orient	Reflexion	
Act	Toolformer	CodeAct Agentless*
全循环	ReAct（建立 T-A-O 循环本身）	
评测	SWE-bench（定义端到端评测基准）	
多 Agent	MetaGPT / AutoGen / OpenHands	

* Agentless 同时覆盖 L-R-V 三阶段，但以固定流水线而非循环的方式

当前位置：“全循环 / ReAct”——我们刚刚理解了循环本身。接下来的课程将依次进入这张图的各个区域。

1.6 Boyd 定律在 AI Agent 中的推论

回到 Boyd。他的核心命题——“循环转速决定胜负”——在 AI 编程 Agent 领域产生了三条具体推论，它们贯穿整门课程：

推论一：基础能力是门槛，循环机制是放大器。

一个小模型跑 5 轮 ReAct 循环修 bug 的成功率，远高于同一个模型一次性开环生成修复 patch 的成功率。模型参数没增加一个，但系统能力跃升了。这不是工程 trick，这是 Boyd 理论的直接推演——中等能力 + 快速循环 > 高能力 + 无循环。

推论二：感知设计（Observe）决定循环质量的上限。

F-86 的气泡座舱 = SWE-agent 的 ACI 设计。如果 Agent 每次只能看到噪声满满的 cat 全文输出，它的 Orient 环节再强也无济于事——垃圾进、垃圾出。精心设计的工具接口（Read 的 offset/limit、Grep 的 head_limit、文件树的 50 行上限）控制了信息粒度，让模型的“视野”始终聚焦在关键信息上。

推论三：协调开销决定编队 OODA 的上限。

Boyd 从单机空战扩展到编队作战时发现：编队的整体 OODA 转速取决于协调成本。如果每个动作都需要全队沟通确认，循环就慢下来了。Claude Code 的 Agent Team 选择星型拓扑（子 Agent 只与主 Agent 通信，不互相对话），正是在最小化协调开销，保持编队循环速度——这是后面 Agent Team 模块的核心设计逻辑。

这三条推论分别对应课程的三大模块。它们不是独立的知识点，而是 Boyd 理论在不同尺度上的投影。

从这里开始，我们进入具体的技术机制。下一节的问题是：Agent 调用工具的背后，到底是什么协议在运作？模型是怎么“伸出手”去操作环境的？